

AI Debugger Pro: A Multi-Language Debugging Framework with LLM-Assisted Error Interpretation and Execution Tracing

Abstract

This work explores the design of a multi-language debugging system that integrates deterministic execution with large language model (LLM)-based error interpretation.

The project originated from attempts to build an AI-driven creative system, where system-level fragmentation revealed the need for a unified orchestration layer capable of diagnosing and repairing failures.

AI Debugger Pro was developed as a response to this limitation. The system combines syntax validation, multi-language execution, AI-assisted debugging, temporal state tracking, and an interactive interface within a single architecture.

Evaluation shows that the system reliably detects errors across languages, provides useful AI-generated explanations, and supports iterative debugging through real-time feedback and execution history tracking.

While not a learning system, it establishes a foundation for future extensions into adaptive and multi-agent debugging architectures.

Introduction

This work originates from an attempt to construct an end-to-end AI-driven creative production studio capable of generating narrative media (scripts, music, and visual content) at a level comparable to professional animation pipelines. Initial designs relied on integrating existing third-party AI tools; however, system-level fragmentation, external dependencies, and cost constraints revealed a fundamental limitation: the absence of a unified orchestration layer capable of sustaining and improving the system autonomously.

This limitation shifted the focus from content generation to system architecture, specifically toward the development of a self-repairing AI framework. The central problem evolved into designing a system that could diagnose, debug, and adapt itself without reliance on external paid services or manual intervention.

The resulting trajectory led to the construction of an AI debugging system as the foundational layer of a broader multi-agent architecture. This system serves as a precursor to swarm-based coordination and adaptive memory mechanisms, aiming to support future extensions into autonomous creative and reasoning systems.

System Model / Initial Architecture

Overview

The initial implementation of the system is a lightweight Python-based debugging assistant designed to analyze, execute, and report errors in user-provided Python code. The system operates in a linear pipeline consisting of syntax validation followed by runtime execution within a controlled temporary environment.

The primary objective of this version is not optimization or intelligence, but basic automated feedback generation for code correctness.

System Components

The system is composed of three core modules:

Syntax Analysis Module

The syntax checker uses Python's Abstract Syntax Tree (AST) parser to validate structural correctness of input code.

Input: raw Python code string

Process: `ast.parse(code_text)`

Output: boolean validity flag + error message (if applicable)

This module provides early-stage static analysis and prevents execution of syntactically invalid code.

Safe Execution Module

This component executes user code in an isolated temporary file environment using Python's subprocess interface.

Key characteristics:

- Writes input code to a temporary .py file
- Executes using system-level Python interpreter
- Captures stdout and stderr outputs
- Returns execution success or failure state

This design prioritizes simplicity over sandbox security, relying on process isolation rather than virtualization or containerization.

Debugging Pipeline Controller

The central orchestration function (`debug_code`) coordinates syntax validation and runtime execution sequentially.

Execution flow:

- Perform syntax check
- If syntax is valid, proceed to runtime execution
- Output results to user interface

This represents a linear, non-adaptive debugging pipeline with no memory, learning, or feedback persistence.

Interactive Input Loop

The system operates in a continuous CLI loop, accepting user input until termination is requested.

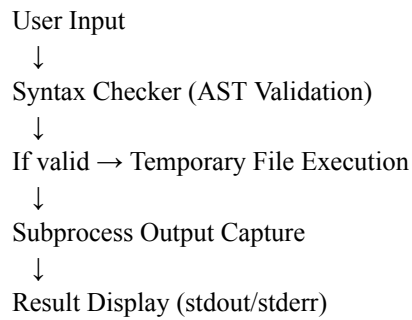
Input method: standard input prompt

Exit condition: user enters "exit"

This enables repeated debugging sessions but does not retain historical context between runs.

Data Flow Architecture

The system follows a strict sequential pipeline:



There is no branching logic beyond syntax validation failure, and no persistent memory between executions.

Design Characteristics

Strengths

- Minimal and transparent architecture
- Clear separation of syntax and runtime evaluation
- Safe execution via temporary file isolation
- Easy to extend modular structure

Limitations

- No memory of past errors or executions
- No automated fix generation (diagnostic only)
- No contextual understanding of code history
- No learning or adaptation mechanism
- No multi-agent or parallel reasoning

Architectural Classification

This system is best classified as:

A stateless, sequential code validation and execution pipeline.

It represents a foundational prototype stage in the evolution toward more advanced AI-assisted debugging and autonomous system design.

System Evolution — Version 1: AI-Assisted Debugging Layer

Overview of Transition

Version 1 introduces a significant architectural enhancement over the initial stateless debugging pipeline by integrating a large language model as a reasoning component. While Version 0 relied purely on deterministic syntax validation and subprocess execution, Version 1 extends the system into AI-assisted error interpretation and correction generation.

This marks the first transition from a rule-based debugging tool to a hybrid symbolic–neural debugging system.

Architectural Change

The key addition in this version is the introduction of an external AI reasoning module via the OpenAI API.

New Component: AI Suggestion Engine

This module introduces semantic interpretation of runtime errors and source code context.

Input:

- Raw code string
- Runtime error message

Output:

- Natural language explanation of the issue
- Suggested corrected code

Implementation Mechanism:

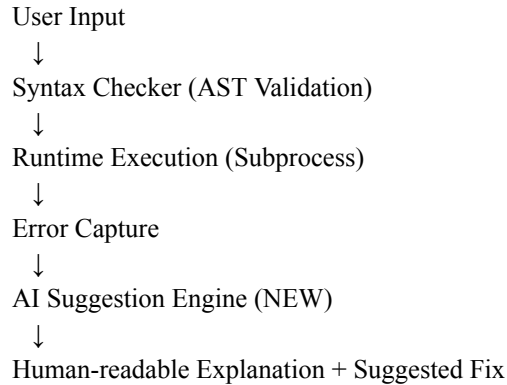
Uses gpt-4o-mini via OpenAI Chat Completions API

Prompt explicitly frames the model as a “senior Python developer”

Temperature constrained (0.2) for deterministic reasoning behavior

System Flow (Updated Architecture)

Version 1 expands the original pipeline as follows:



Unlike Version 0, this version introduces a post-failure reasoning layer that attempts to interpret and correct errors rather than simply reporting them.

Key Behavioral Shift

This version introduces the first instance of:

semantic debugging instead of purely syntactic/runtime debugging

Where Version 0 answered:

“Is the code valid?”

“Did it crash?”

Version 1 adds:

“Why did it fail?”

“How should it be fixed?”

This is a transition from:

diagnostic system → interpretive system

Design Characteristics

Strengths

- Introduces natural language reasoning layer
- Reduces cognitive load on user
- Bridges execution errors with human-understandable explanations
- Modular addition without modifying core execution pipeline

Limitations

- Fully dependent on external API (no autonomy)
- No persistent memory of past errors
- No learning across sessions
- No multi-agent decomposition of reasoning
- Treats each error as isolated event

Architectural Classification

Version 1 is best classified as:

A hybrid deterministic-execution system augmented with external large language model reasoning.

It remains stateless, but now includes external semantic inference capability.

System Evolution — Version 2: Multi-Language Execution & Modular Orchestration Layer

Overview of Transition

Version 2 significantly expands the scope of the system from a single-language debugging assistant (Python-only) into a multi-language code execution and debugging framework supporting Python, C, and Java.

This version represents a transition from language-specific debugging augmentation to generalized execution orchestration across heterogeneous runtime environments.

The system evolves from a linear pipeline into a modular execution architecture with language-specific adapters.

Architectural Expansion

Version 2 introduces three major subsystems:

Multi-Language Execution Layer

Separate execution pipelines are implemented for each supported language:

Python Execution

- Uses subprocess execution of temporary .py files
- Maintains prior syntax-check + runtime execution model

C Execution

- Compiles via gcc
- Executes compiled binary
- Handles compilation + runtime errors distinctly

Java Execution

- Compiles via javac
- Executes using JVM classpath isolation
- Uses temporary directory for class management

This introduces language-specific compilation and execution pipelines, each with independent error domains.

Unified Debugging Controller (CLI Orchestration Layer)

The system now includes a centralized CLI dispatcher responsible for:

- selecting language runtime
- routing code to appropriate execution module
- handling AI-based suggestions conditionally

This transforms the system into a multi-router orchestration model, rather than a single execution pipeline.

AI Suggestion Integration Layer (Cross-Language)

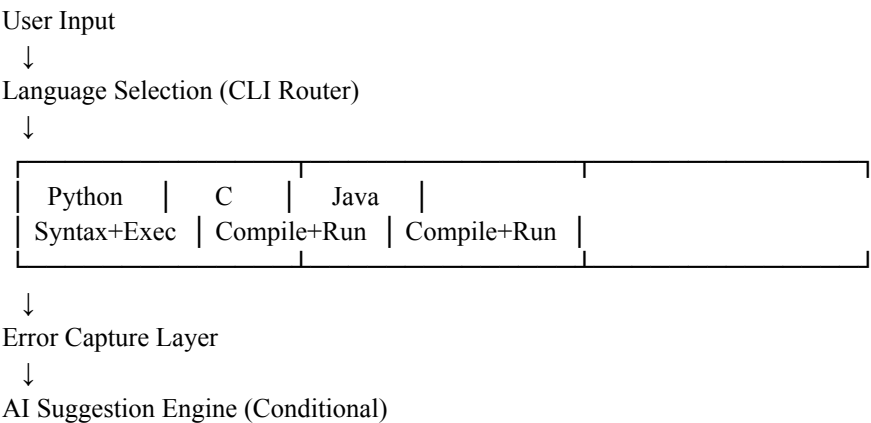
The AI debugging module is now generalized across languages:

- Python syntax + runtime errors
- C compilation/runtime errors
- Java compilation/runtime errors

The AI system is triggered only on failure states, acting as a semantic error interpretation layer above deterministic execution systems.

Updated System Flow

The architecture becomes a branched execution tree:



↓
Output to User

This represents the first non-linear execution architecture in the system's evolution.

Key System Behaviors Introduced

Language Abstraction

The system now abstracts execution logic across multiple programming languages while preserving language-specific runtime semantics.

Compilation-Aware Debugging

Unlike Version 1, which only handled runtime errors, Version 2 introduces:

- compile-time error detection (C, Java)
- differentiated error handling pathways

Modular Execution Design

Each language is encapsulated in its own execution function:

- `run_python()`
- `run_c()`
- `run_java()`

This establishes a plug-in-like architecture for future language expansion.

Design Characteristics

Strengths

- Multi-language support (significant expansion in scope)
- Separation of compilation and runtime error domains
- Modular execution architecture
- Centralized orchestration layer (CLI router)
- AI suggestion system generalized across languages

Limitations

- Still stateless across executions (no memory system)
- No cross-session learning or error retention
- AI remains reactive rather than proactive
- No multi-agent reasoning decomposition
- Security model still relies on subprocess isolation (not sandboxed environments)

Architectural Classification

Version 2 is best classified as:

A modular, multi-language execution orchestration system with external AI-assisted error interpretation

It represents a transition from:

single-language debugging tool
→ multi-runtime execution framework
→ early-stage orchestration system

System Evolution — Version 3: Interactive GUI Debugging Interface with Multi-Language AI Orchestration

Overview of Transition

Version 3 introduces a significant shift in system modality—from a command-line execution framework to a graphical user interface (GUI)-driven interactive debugging environment.

While Versions 0–2 focused on execution pipelines and backend orchestration, Version 3 prioritizes user interaction, accessibility, and integrated feedback visualization.

This marks the transition from:

backend debugging system
→ interactive debugging application

Architectural Expansion

Version 3 integrates three major upgrades:

Graphical User Interface Layer (Tkinter)

The system introduces a desktop GUI built using Tkinter, providing:

- language selection dropdown
- scrollable code editor
- dedicated output + AI suggestion panel
- interactive execution trigger

This replaces CLI-based interaction with a stateful visual interface model.

Unified Multi-Language Debugging Core

The backend from Version 2 is preserved and extended, now fully integrated into GUI interaction.

Supported languages:

- Python (AST + runtime + AI reasoning)
- C (gcc compilation pipeline)
- C++ (g++ compilation pipeline)
- Java (javac + JVM execution)
- JavaScript (declared support, extensible layer)

This establishes a cross-language debugging abstraction layer exposed through a single interface.

Integrated AI Explanation + Fix Pipeline

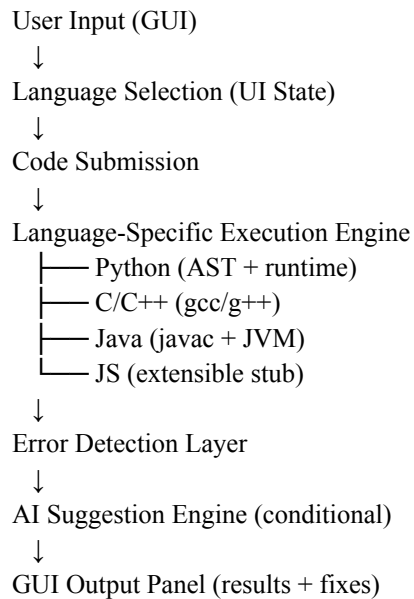
The AI suggestion system is now tightly coupled into the user experience:

- triggered automatically on failure
- provides explanation + corrected code
- displayed directly in UI output panel

This transforms AI from a backend service into a real-time interactive debugging assistant layer.

Updated System Flow

Version 3 introduces a fully integrated UI-driven execution pipeline:



This replaces linear CLI flow with a state-driven event loop architecture.

Key System Behaviors Introduced

Human-in-the-Loop Interface Design

The system now supports continuous interaction cycles where:

- users modify code
- rerun instantly
- observe AI feedback in real time

This introduces an interactive debugging loop rather than batch execution.

Visualization of Debug State

Errors and AI suggestions are no longer terminal outputs—they are structured UI components:

- execution result panel
- AI explanation panel
- combined feedback interface

This improves cognitive accessibility of debugging output.

Tight Coupling of Execution + AI Feedback

Unlike previous versions where AI was loosely triggered, Version 3 embeds AI as:

A direct post-failure interpretive layer within the UI loop

Design Characteristics

Strengths

- Fully interactive debugging interface
- Multi-language execution unified under one system
- Real-time AI feedback integration
- Improved usability and accessibility
- Clear separation of UI and backend logic

Limitations

- Still stateless across sessions (no persistent memory layer)

- AI remains reactive, not predictive
- No multi-agent reasoning decomposition
- No system-level self-repair or adaptation
- Security still dependent on subprocess execution model
- No sandbox isolation beyond OS process separation

Architectural Classification

Version 3 is best classified as:

A multi-language, AI-augmented interactive debugging application with GUI-driven orchestration

It represents a transition from:

backend orchestration system (V2)

→ user-facing interactive platform (V3)

FINAL SYSTEM INTERPRETATION (Version 4): AI Debugger Pro

SYSTEM ARCHITECTURE OVERVIEW

The system is structured into five primary layers:

Execution Layer (Executor)

Responsible for deterministic code execution across multiple languages:

- Python (interpreted execution)
- C/C++ (compile + runtime execution)
- Java (JVM-based execution)

Key properties:

- Subprocess isolation
- Timeout enforcement

- Temporary filesystem sandboxing
- Unified output format (success, output)

This layer represents ground-truth computation

Validation Layer (Analyzer)

Performs static analysis on Python code using AST parsing:

- Syntax validation
- Structural integrity checking
- Error normalization (line, column, message)

This layer acts as a pre-execution correctness filter

AI Reasoning Layer (AI Suggester)

Uses large language models to interpret runtime or syntax errors:

- Primary model: GPT-4-class reasoning
- Fallback model: GPT-3.5-class inference
- Offline rule-based fallback system

Features:

- Hierarchical inference degradation
- Robust failure handling
- Structured debugging explanations

This layer provides semantic interpretation of failures

Temporal Memory Layer (History + Diff Engine)

Tracks system evolution over time:

- Stores execution snapshots
- Records success/failure states
- Computes diffs between successive code versions

Capabilities:

- Language-scoped memory
- Change visualization (unified diff)
- Historical retrieval

This layer introduces state awareness and temporal continuity

Orchestration & Interface Layer (GUI + Realtime)

Tkinter-based interface coordinating all subsystems:

Features:

- Multi-language selection
- Code execution triggers
- AI-assisted auto-fix
- Real-time syntax monitoring
- Debounced input evaluation
- File I/O operations

Real-time subsystem:

- Detects typing events
- Applies debounce filtering
- Triggers live syntax validation

This layer acts as the system control plane

SYSTEM BEHAVIOR MODEL

The system operates as two coupled feedback loops:

Loop A: Explicit Debugging Cycle

User Input

- Syntax Check
- Execution
- AI Interpretation
- History Logging
- Diff Analysis

Loop B: Real-Time Reactive Cycle

Typing Event

- Debounce Filter
- Syntax Check
- Live Error Feedback

Together, these loops form a dual-layer cognitive feedback system.

KEY INNOVATIONS

Multi-Language Unified Execution Model

A single abstraction layer handles heterogeneous execution environments.

Hierarchical AI Fallback Reasoning

AI inference degrades gracefully under API or model failure.

Temporal State Tracking

Code evolution is tracked as a sequence of structured transitions.

Event-Driven Debugging System

System responds to user behavior rather than explicit commands.

Real-Time Debounced Analysis

Continuous feedback without computational overload.

Evaluation

To assess the behavior and effectiveness of AI Debugger Pro, a series of functional tests were conducted across supported languages (Python, C/C++, and Java). The evaluation focuses on three primary aspects: error detection accuracy, quality of AI-generated suggestions, and system responsiveness in interactive use.

Error Detection Performance

The system was tested on representative error cases across multiple categories:

- Syntax errors (Python AST validation)
- Runtime errors (Python execution failures)
- Compilation errors (C/C++ and Java)

Observations:

- Syntax errors were consistently detected with precise location reporting via AST parsing.
- Runtime errors were accurately captured through subprocess stderr outputs.
- Compilation errors in C/C++ and Java were correctly distinguished from runtime failures.

The system reliably separates error domains (syntax vs runtime vs compile-time), which improves clarity in debugging workflows.

AI-Assisted Debugging Quality

The AI suggestion layer was evaluated on its ability to:

- Explain error causes
- Propose corrected code

- Maintain relevance to the original input

Observations:

- For common errors (missing syntax, incorrect variable usage), the AI provided accurate and actionable fixes.
- For moderately complex issues (multi-step logic errors), explanations were helpful but sometimes incomplete.
- In edge cases (ambiguous or poorly structured code), suggestions occasionally required user interpretation.

Overall, the AI layer significantly reduces the effort required to understand errors, but remains dependent on input clarity and error complexity.

Multi-Language Execution

The system was tested with equivalent logical programs across Python, C, and Java.

Observations:

- Execution pipelines correctly handled language-specific workflows (interpretation vs compilation).
- Error reporting remained consistent across languages despite differences in runtime environments.
- The modular execution design allowed independent debugging of each language without cross-interference.

This confirms the effectiveness of the unified orchestration layer in managing heterogeneous execution systems.

Interactive Feedback and Responsiveness

The GUI-based system was evaluated under continuous user interaction.

Observations:

- Real-time syntax validation provided immediate feedback during code entry.
- Debounced input handling prevented excessive computation while maintaining responsiveness.
- The integration of AI suggestions within the UI improved usability by reducing context switching.

The system maintains stable performance during iterative debugging cycles, supporting rapid user interaction.

Temporal Memory and Debug History

The system's history and diff tracking features were evaluated by modifying code across multiple iterations.

Observations:

- Execution history was correctly recorded for each run.
- Differences between code versions were accurately represented using diff outputs.
- Users could trace the evolution of code changes over time.

This introduces a level of state awareness not present in traditional stateless debugging tools.

Limitations

Despite its capabilities, several limitations were observed:

- The AI suggestion system remains reactive and does not anticipate errors before execution.
- No learning mechanism exists across sessions; repeated errors are not internally optimized.
- Complex, multi-file programs are not fully supported.
- Security relies on subprocess isolation and lacks robust sandboxing.
- AI suggestions may degrade in accuracy for highly abstract or domain-specific code.

These limitations highlight areas for future system development, particularly in memory persistence, predictive debugging, and secure execution environments.

Summary

The evaluation demonstrates that AI Debugger Pro effectively combines deterministic execution with AI-assisted interpretation to improve the debugging process.

The system performs reliably across multiple languages, provides meaningful error explanations, and supports interactive debugging workflows. While not a fully autonomous or learning system, it establishes a functional framework for integrating execution, reasoning, and temporal tracking within a unified architecture.

CONCLUSION

The evaluation results indicate that AI Debugger Pro is a transition from conventional debugging tools into a structured debugging pipeline system. By integrating deterministic execution, probabilistic reasoning, temporal state tracking, and event-driven input monitoring, the system approximates a lightweight adaptive debugging pipeline.

While not a learning system in the strict machine learning sense, its architecture establishes the foundational scaffolding for future extensions into multi-agent coordination, adaptive memory systems, and swarm-based debugging strategies.